

スキルシート

同じ内容を 2 つの形式で公開 — [Markdown](#) (GitHub 原本) ・ [Web](#) (目次つき)

基本情報

- 拠点: 東京都江戸川区
- 現在のポジション: [株式会社ZENSHIN](#) CTO
- エンジニア歴: 10年目 (2017年～)
- 外部リンク:
 - GitHub: <https://github.com/sugurutakahashi-1234>
 - X: https://x.com/suguru_takaha4
 - Qiita: <https://qiita.com/sugurutakahashi12345>
 - Zenn: <https://zenn.dev/ikuraikura>

強み

- 創業フェーズの CTO 経験
 - 技術選定から開発・運用・社内インフラ整備まで、事業に必要なものを自分で決めて実装
 - サービス 3 つとマッチング 2 サイトを、企画から公開・運用まで担当
 - 小規模でも維持できる構成の選定 (Cloudflare 中心に寄せ、運用コストを抑制)
- AI システム開発
 - LLM で人材マッチングと案件評価を自動化するシステムを設計から運用まで開発
 - Cloudflare 上に RAG・ベクトル検索・AI エージェントを組み込んだアプリケーションを構築
 - 非エンジニアが Claude Code / Codex で操作できるよう、社内システムを MCP 前提で設計
 - AI の判定と人間の判断を記録して定期実行で点検し、プロンプトを更新するループの構築
- FDE としての AI 活用支援
 - 経営層から実装担当者まで、相手に合わせた AI 活用の相談・提案
 - 案件獲得の営業・商談に加え、顧客の業務に合わせた AI ツールの実装・導入も担当
- 0 → 1 リード経験 (受託 4 件 + CTO 1 社)
 - アーキテクチャ選定・CI/CD 構築・チーム運用の設計など、立ち上げの技術面を担当
 - 創業直後のスタートアップでは、1 人目のエンジニアとしてプロダクトと社内システムを構築
 - 受託の内訳: iOS リード 3 件、Flutter リード 1 件
- フルスタックの実装力
 - Swift・React Native・Flutter で iOS / Android、Next.js・Astro で Web を実案件で実装
 - Bun・Hono・oRPC・Drizzle・TanStack Start を本番で運用
 - マルチクラウド (GCP / Cloudflare / AWS) の構成を Terraform で IaC 化し、運用まで担当
- ビジネス KPI 起点での開発経験
 - 獲得コストと LTV を起点に、課金層と無課金層それぞれの収益施策を設計・実装
 - 施策立案から 1 週間でリリースする PDCA を回し、効果が出ない施策の削除も判断
- OSS 開発・技術発信
 - Clean Architecture を基にした設計手法 [Framework-Independent Architecture](#) を考案・公開
 - [ai-chat-md-export](#) など TypeScript 製 CLI ツール 4 本を npm / Homebrew で配布
 - [Zenn](#) と [ZENSHIN の技術ブログ](#) で、AI 活用やプロダクト開発の知見を記事・スライドで発信

職務経歴

フリーランス (2021年7月～現在)

- [No.11] [株式会社ZENSHIN](#) (IT コンサルティング / Sler / SES)
 - CTO / Cloudflare・RAG・MCP / 自社プロダクト開発と FDE としての顧客支援
- [No.10] ショートドラマアプリ開発 (エンタメ業界 G 社)
 - フロントエンドエンジニア / React Native (Expo)・Next.js / iOS・Android・Web と KPI 施策

- [No.9] NFT ゲームアプリ開発 (WEB3 特化 Sler F社)
 - Flutter リードエンジニア / Flutter ・ Riverpod / 0 → 1 立ち上げ
- [No.8] SNS アプリ開発 (Sler E社)
 - iOS リードエンジニア / Swift ・ SwiftUI / 0 → 1 立ち上げ
- [No.7] マーケティングリサーチアプリ開発 (スタートアップ D社)
 - iOS リードエンジニア / Swift ・ SwiftUI ・ Amplify / 0 → 1 立ち上げ
- [No.6] ファンクラブアプリ開発 (メガベンチャー C社)
 - iOS エンジニア / Swift ・ UIKit ・ Redux / スタンダライズ・景品交換機能の開発
- [No.5] ドローン制御アプリ開発 (ドローンベンチャー B社)
 - iOS エンジニア / Swift ・ SwiftUI ・ VIPER / 飛行の安定性改善と複数社ドローン対応

外資系ITコンサル A社 (2017年4月～2021年6月)

- [No.4] ショッピングアプリ開発
 - Flutter エンジニア / Flutter ・ Google Maps API / クロスプラットフォームのデモアプリ開発
- [No.3] 飲食店管理アプリ開発
 - iOS リードエンジニア / Swift ・ SwiftUI ・ MVVM / BtoB 向け MVP 開発
- [No.2] クレジットカードアプリ iOS 開発
 - iOS エンジニア / Swift ・ UIKit ・ MVVM / 見積もり・実装・テスト・レビュー
- [No.1] クレジットカードアプリ API 開発
 - サーバーサイドエンジニア / PHP (CodeIgniter) ・ Java / リニューアルに伴う API・バッチ開発

案件詳細

[No.11] 株式会社ZENSHIN (IT コンサルティング / Sler / SES)

チーム体制

- CTO として、顧客企業の AI 活用支援と社内プロダクト・インフラの開発・運用を一人で担当

案件概要・担当業務

- 創業フェーズの株式会社ZENSHIN の CTO として、以下 6 つの業務を並行して推進
 - **[業務1] 自社サービスの企画・立ち上げ:** 提供サービス 3 つとマッチング 2 サイトの公開
 - **[業務2] AI 活用支援 / 提案活動 (FDE) :** AI 活用の相談・提案から実装・導入まで支援
 - **[業務3] AI マッチングシステム開発:** Cloudflare での RAG / MCP / AI エージェント基盤
 - **[業務4] AI 案件選別システム開発:** 外部案件を LLM で自動評価する社内システム
 - **[業務5] [ホームページ](#)・[技術ブログ](#):** 設計・構築・運用
 - **[業務6] 社内インフラ管理:** Terraform によるマルチクラウド IaC

経験した技術

- **FDE / AI 活用支援 (業務2)**
 - Claude Design や v0 による商談前の高速プロトタイピング
 - LLM ツールの使い分け指導、プロンプト改善、MCP 化・Skills 設計の方針策定
 - 建設業向けに、過去事例・基準類・図面から施工計画書の下書きを生成する仕組みを開発
- **RAG / ベクトル検索マッチング (業務3)**
 - Workers AI (bge-m3) + Vectorize による意味ベースの類似検索
 - ベクトル検索 (一次絞り込み) → AI エージェント採点 (二次精査) の多層パイプライン設計
 - 自作の評価スクリプトで LLM モデルを比較検証し、シーンごとに使い分け
- **MCP サーバー / AI エージェント運用 (業務3)**
 - 70 以上のツールを持つ MCP サーバーの開発 (OAuth 認証・マルチテナント認可)
 - WebMCP (ブラウザ内 MCP) の Origin Trial 先行導入
 - 50 以上のスキルと定期実行の設定による業務自動化 (Claude Code / Codex 両対応)
- **LINE 公式アカウント基盤 (業務3)**
 - 友だち紐付け不要の本人アカウント連携と、スタッフ向けの双方向チャット

- Quick Reply による稼働状況の定期ヒアリング
- **Cloudflare フルスタック（業務1・3・4・5）**
 - Workers / Workflows / Queues / D1 / Vectorize / Browser Rendering によるサーバーレス構成
 - Hono + oRPC + React 19 / TanStack Start によるマッチング 2 サイトと社内 Web アプリ
 - Astro によるホームページと技術ブログ
 - Email Sending ・ Slack Interactivity による採用応募対応の自動化
- **マルチクラウド IaC（業務6）**
 - GCP / Cloudflare / AWS / Google Workspace を Terraform で横断管理
 - Workload Identity Federation (OIDC) による鍵レス認証

取り組み・貢献

- **自社サービスの立ち上げ（業務1）**
 - AI 活用伴走・業務変革コンサルティング・スポット開発の 3 サービスを企画
 - [ZENSHIN フリーランス](#) (toC) と [ZENSHIN エンジニアマッチ](#) (toB) を公開
- **顧客の業務と体制に合わせた提案・納品（業務2）**
 - 決裁者と実際に使う担当者の双方にヒアリングし、業務を観察してから要件を固めて受注
 - 顧客が契約済みの AI（ChatGPT・Gemini など）と利用状況に合わせて構成を選定
- **営業が AI エージェントで実業務を回す状態の実現（業務3）**
 - IT 知識のない営業が Claude Code から自然言語で要員検索～人材紹介を完結
 - 案件 1,000 件 × エンジニア 1,000 名の規模から、5 分以内に適切なマッチングを提示
- **AI の判断が顧客接点まで届く自動ループの構築（業務3・4）**
 - 案件収集 → AI 採点 → 本人 LINE への自動提案 → 本人回答まで、人手を介さない運用を実現
 - AI エージェントの返信・提案には人間の承認と監査記録を必須化
- **AI を過信しない設計・運用（業務3・4）**
 - 条件ミスマッチはサーバー側で決定論的にスコア上限を強制し、AI の過大評価を防止
 - AI の判定と人間の判断のずれを本番データで分析し、プロンプトを 25 回以上改修
 - 構造化は決定論的な変換に任せるなど、AI と機械的な処理の使い分けを徹底
- **技術発信によるブランディング（業務5）**
 - 技術ブログを Astro で立ち上げ、AI 活用やプロダクト開発の知見を記事とスライドで公開
 - 構造化データ・sitemap・llms.txt などの SEO 施策を入れ、Lighthouse 100 を維持
- **一人で運用できる仕組みの整備（業務6）**
 - 社内インフラを徹底的に IaC 化し、コード変更 → PR → plan CI → apply のフローを確立
 - 採用しなかった選択肢を含め、技術的な意思決定の根拠をリポジトリに記録する文化を形成

開発環境

- **言語 / ランタイム**
 - TypeScript , Bun , Python (uv)
- **フロントエンド**
 - Astro , React 19 , TanStack Start / Router / Query , shadcn/ui , Tailwind CSS v4
- **バックエンド / AI システム**
 - フレームワーク: Hono , oRPC , Drizzle ORM , Zod , Better Auth
 - MCP: @modelcontextprotocol/sdk v2 , WebMCP
 - 外部 API 連携: LINE Messaging API , Slack API , Google Drive API , GitHub App
- **クラウド / インフラ**
 - Cloudflare: Workers AI , AI Gateway , Vectorize , Workflows , Queues , Browser Rendering , Email Sending
 - マルチクラウド / IaC: GCP , AWS , Terraform , tflint , dotenvx , mise
- **CI/CD**
 - GitHub Actions , release-please , Wrangler , Renovate
- **テスト / 品質**

- Vitest , Playwright , Storybook , MSW , @axe-core/playwright , oxlint , oxfmt , knip , lefthook
- AI ツール (提案活動・開発)
 - Claude Code , Codex , v0 , Chrome DevTools MCP
- 分析 / 開発ツール
 - Google Search Console , PageSpeed Insights , PostHog , Cloudflare Analytics , VSCode

[No.10] ショートドラマアプリ開発 (エンタメ業界 G社)

チーム体制

- 案件全体人数 : 6名
 - フロントエンド (iOS + Android + Web) : 2名 (担当)

案件概要・担当業務

- 0 → 1 直後のグロースフェーズにおけるショートドラマアプリの開発
- React Native (Expo) による iOS / Android 同時開発を担当し、並行して Next.js による Web 版の開発も実施

経験した技術

- **React Native (Expo) / Next.js**
 - Monorepo 構成による iOS / Android / Web のクロスプラットフォーム開発
 - Expo SDK のメジャーバージョンアップ対応 (v52→v53、v53→v54、v54→v55)
 - Solito によるモバイル (Expo) / Web (Next.js) 間のコード共有
 - Tamagui を用いたプラットフォーム間の UI 統一
 - expo-iap によるアプリ内課金の実装
 - Rive を用いたアニメーションの組み込み
 - Next.js による Web アプリケーション開発
 - TanStack Query によるデータフェッチング・キャッシング
 - Crisp SDK を用いたカスタマーサポート用チャット機能の実装
- **開発プロセス改善**
 - Storybook による UI コンポーネントカタログの作成
 - Maestro によるモバイルアプリでの E2E テストの導入
 - mise による開発環境のセットアップ簡略化とバージョン統一
- **AI・MCP 活用**
 - Figma MCP を活用した Figma デザインからの実装
 - Storybook でコンポーネントを作成し、Chrome DevTools MCP / Playwright MCP で Claude Code による検証
 - GitHub Actions による Claude Code 自動レビューの仕組みの構築

取り組み・貢献

- **獲得コストと LTV を起点としたグロース施策**
 - 獲得単価に課金 LTV が届かない前提を CEO と共有し、LTV を上げる施策と一緒に設計
 - CEO の数値分析から施策の意図を読み取り、アプリの画面と導線への落とし込みを担当
 - 課金導線・ガチャ・ミッション機能・無料開放を実装し、サブスク契約者数と ARPU を改善
 - 課金転換しない層からはリワード広告・オフアールで収益化
- **高速な PDCA サイクルの実践**
 - 施策立案から 1 週間でリリースする PDCA を、スプリントごとに反復
 - A/B テストで、季節要因や曜日パターンなど複数の変数がある中で施策単体の効果を測定
 - 効果が見込めない施策はすぐに削除し、効果がないという事実も意思決定に反映
- **デザイナーとの協業**
 - 表示内容や画面遷移の導線、コンポーネントの色や配置といった UI/UX の仕様にまで踏み込んで改善を提案
 - 仮実装を素早く共有し、デザイナーと議論しながらリリース前に改善
 - リリースごとの KPI をデザイナーと共有し、データに基づく改善提案を反復

開発環境

- **React Native (Expo) / Next.js**

- アーキテクチャ: Monorepo
 - 主要ライブラリ: TanStack Query , ts-proto , Tamagui , Solito , Expo Modules API
 - コード品質・テスト: Storybook , ESLint , Prettier , Vitest , Maestro
- **Firebase**
 - Authentication , App Distribution , Remote Config , Analytics
- **Google Cloud**
 - Cloud Run
- **分析ツール**
 - BytePlus DI , Looker Studio , Adjust
- **CI/CD**
 - GitHub Actions
- **プロジェクト管理**
 - GitHub Projects , Notion , Linear
- **開発ツール**
 - VSCode , Android Studio , Xcode , GitHub Copilot , Claude Code , Codex , Gemini
- **デザインツール**
 - Figma

[No.9] NFT ゲームアプリ開発 (WEB3 特化 Sler F社)

チーム体制

- 体制
 - PdM : 1名
 - PM : 1名
 - デザイナー : 1名
 - サーバーサイドエンジニア : 2名
 - Flutter エンジニア : 1名 (担当)

案件概要・担当業務

- 位置情報を使った NFT ゲームアプリの 0 → 1 立ち上げ案件
- 唯一の Flutter エンジニアとして、アーキテクチャ考案・ライブラリ選定から実装までを担当
- PM・デザイナー・サーバーサイドチームとの仕様調整も担当

経験した技術

- **Flutter**
 - Riverpod と Hooks を用いた状態管理
 - Google Maps API と google_maps_flutter , geolocator による地図表示と位置情報の取得
 - go_router を用いた画面遷移の実装
 - openapi_generator を用いた API クライアントコードの自動生成
 - dio を用いた HTTP 通信およびインターセプターによる JWT 認証の実装
 - flutter_secure_storage を用いたセキュアなデータ保存
 - permission_handler を用いた位置情報取得、写真撮影、写真フォルダへのアクセスの実装
 - slang による多言語対応
 - pedantic_mono によるコード品質の向上
 - ThemeData によるデザインシステムの実装
 - fvm による Flutter バージョンの管理
- **開発プロセス改善**
 - Prism を活用した API モックサーバーの構築
 - Lefthook による pre-commit 時の静的解析実行

取り組み・貢献

- **開発体制の改善活動**
 - GitHub Projects のスクラムボードで、タスクの進捗を可視化

- デイリーミーティングで、メンバー間の情報共有と開発プロセスの改善を推進
 - バグの発見から修正までのプロセスを整備し、チーム内で運用
- **API インターフェース設計と UI 先行開発**
 - サーバーサイドの Pull Request をレビューし、開発段階で API インターフェースの改善点をフィードバック
 - OpenAPI の yaml から Prism でモックサーバーを整備し、UI の先行開発を実施
- **デザインシステムの導入**
 - デザイナーと協力してデザインシステムを設計し、デザインの一貫性を確立
 - デザインシステムを Flutter の `ThemeData` で定義し、UI の実装コードを削減

開発環境

- **Flutter**
 - アーキテクチャ: `Riverpod`, `Hooks`
 - 主要ライブラリ: `go_router`, `dio`, `slang`, `permission_handler`, `flutter_secure_storage`, `pedantic_mono`, `freezed`, `google_maps_flutter`, `geolocator`, `openapi_generator`, `fvm`
- **CI/CD**
 - `GitHub Actions`
- **プロジェクト管理**
 - `GitHub Projects`, `Notion`, `Slack`
- **開発ツール**
 - `VSCode`, `Android Studio`, `Xcode`, `GitHub Copilot`, `ChatGPT`
- **デザインツール**
 - `Figma`

[No.8] SNS アプリ開発 (Sler E社)

チーム体制

- 案件全体人数: 約10名
 - iOS エンジニア: 1名 (担当)

案件概要・担当業務

- 0 → 1 フェーズでの SNS アプリの立ち上げ案件
- 唯一の iOS エンジニアとして、アーキテクチャ考案・ライブラリ選定・CI/CD 構築からすべての実装までを担当
- PM・デザイナー・サーバーサイドチームとの仕様調整も担当

経験した技術

- **Swift**
 - Xcode 16 Beta での Strict Concurrency を含む Swift 6 対応
 - AVFoundation を活用した録音/再生の機能実装
 - [WhisperKit](#), [Speech](#) SDK を活用した音声データの文字起こしの実装
- **開発プロセス改善**
 - [Swift OpenAPI Generator](#) で API 通信処理を自動生成する GitHub Actions パイプライン
 - [Swagger UI Action](#) を用いた API 仕様書の自動生成の GitHub Actions パイプラインの構築
 - [tbls](#) を用いた MySQL のテーブル定義書の自動生成の GitHub Actions パイプラインの構築
 - [pixelmatch](#) による View のスナップショットの差分検出の実装

取り組み・貢献

- **アジャイル開発**
 - テスタブルなアーキテクチャの導入: モック化した API レスポンスで、提供前から実装に着手
 - デバッグ画面の作成: 検証用のデバッグ画面により、新機能や View を早期に検証
 - Docs as Code の導入: [Swagger UI Action](#) や [tbls](#) によるドキュメント生成を調査し、サーバーサイドチームに展開
 - 実装: ワイヤフレーム段階のデザインを基に iOS アプリを実装し、実装・仕様の課題を早期に発見してチームへ共有
- **CI/CD 環境の構築**
 - Xcode Cloud 導入: PR マージをトリガーに TestFlight 配信を自動化し、新機能を迅速に検証

- API インターフェース変更の自動 Pull Request 作成: OpenAPI の変更をトリガーに、iOS リポジトリへ自動で Pull Request を作成する GitHub Actions を構築
- スナップショット差分テスト: View の差分テスト環境を構築し、不具合を早期に発見
- **iOS メンバーの増員や引き継ぎを見越した GitHub 管理**
 - ドキュメント整備: 環境構築手順、ライブラリ選定理由、アーキテクチャ、CI/CD 構成図、ブランチ戦略を README に記載
 - プロジェクト管理: リリースノート・タグ・マイルストーン・GitHub Projects を整備し、タスクの進捗を時系列で追跡
- **Swift 6 移行とテストコードの自動生成**
 - コード生成: View 層・UseCase 層のテストコードを含むボイラープレート [Sourcery](#) や [Mockolo](#) で自動生成
 - Swift 5 → Swift 6 への移行: 早い段階から Beta 版 Xcode で検証し、大きなトラブルなく完了

開発環境

- **Swift**
 - アーキテクチャ: Clean Architecture , Swift Package Manager (マルチモジュール構成)
 - Swift 標準 SDK & API: SwiftUI , Swift Package Manager , Swift Concurrency , Combine , AVFoundation , Speech , Swift Testing , String Catalogs , Swift OpenAPI Generator
- **CI/CD**
 - Xcode Cloud , GitHub Actions , Renovate
- **プロジェクト管理**
 - GitHub Projects , Notion , Backlog
- **デザインツール**
 - Figma

[No.7] マーケティングリサーチアプリ開発 (スタートアップ D社)

チーム体制

- 案件全体人数 : 約15名
 - iOS エンジニア : 3名 (iOS リードエンジニア担当)

案件概要・担当業務

- スタートアップ企業の 0 → 1 フェーズでのマーケティングリサーチサービス立ち上げ案件
- toC 向けコンテンツ配信アプリと toB 向け BI ツールの 2 サービス構成で、iOS チームのリードエンジニアを担当

経験した技術

- **Swift**
 - iOS 16 以上を対象 OS として SwiftUI での画面開発
 - Clean Architecture x Swift Package Manager でのマルチモジュール構成の構築
 - Xcode Cloud での CI/CD 環境の構築
 - Protocol Buffers に対応した [SwiftProtobuf](#) のライブラリを用いたデータ連携
 - async/await, AsyncStream, TaskGroup, Actor などを用いた Swift Concurrency による非同期処理のハンドリング
 - [AWS Amplify SDK](#) を用いた Cognito での SMS での認証・認可、AppSync による GraphQL 疎通、Pinpoint によるログイベント送信、S3 とのデータ連携
 - デザインシステムを活用した画面実装
 - AVFoundation を用いた動画の再生
 - ReplayKit を用いた画面のレコーディング
 - 視線や感情の時系列データの Combine を用いたハンドリング
 - JavaScript を用いたアプリ内 WebView のイベントハンドリング
- **開発プロセス改善**
 - リリース tag・リリースノート・レビュー追加を自動化する GitHub Actions Workflow
 - [Renovate](#) によるライブラリの自動更新 PR の作成の環境構築
 - [Periphery](#) による Swift の未使用コードの静的解析
 - Swift-DocC による iOS アプリのドメイン層のドキュメント化
 - [Mockolo](#) によるテスト用の Mock の自動生成
 - GitHub Copilot, ChatGPT の活用

取り組み・貢献

- 0 → 1 フェーズの iOS リードとして、アーキテクチャ・ライブラリ選定、ブランチ戦略、リリース手順の確立を担当
- CI/CD 環境の構築と、iOS チームのスクラムボード運用の設計
- チームに経験者のいない AWS Amplify SDK や SwiftProtobuf を、サンプルアプリで先行検証して展開し、採用まで推進
- 視線分析・感情分析の SDK を、入れ替えても影響範囲が最小になるアーキテクチャで組み込み
- PdM・デザイナー・サーバーサイド・データ分析と仕様やデータ連携インターフェースを調整
- iOS チームの issue 運用を担い、メンバーのタスクが途切れない状態を維持

開発環境

- **Swift**
 - アーキテクチャ: VIPER , Clean Architecture , Swift Package Manager
 - Swift 標準 SDK & API: SwiftUI , Swift Package Manager , Swift Concurrency , Combine , Swift-DocC , AVFoundation , Core ML , WebKit , ReplayKit , Logger
 - サードパーティ製 SDK: SwiftProtobuf , Firebase , Amplify , Nimble/Quick , LicensesPlugin , PhoneNumberKit , DeviceKit , SwiftFormat , SwiftGen , Lottie , Mockolo , Mint , Periphery
- **クラウド連携**
 - AWS Amplify: AppSync (GraphQL) , Cognito , S3 , Pinpoint
 - Firebase: Crashlytics
- **CI/CD**
 - Xcode Cloud , GitHub Actions , Renovate
- **プロジェクト管理**
 - GitHub Projects , Notion , Backlog
- **インターフェース共有**
 - Protocol Buffers , Swagger
- **デザインツール**
 - Figma

[No.6] ファンクラブアプリ開発（メガベンチャー C社）

チーム体制

- 案件全体人数：約30名
 - iOS エンジニア：5名（担当）

案件概要・担当業務

- アーティストのファンクラブアプリで、スタンプラリー機能と景品交換機能を開発
- デザイナーとの仕様調整、見積もり、実装、レビュー、バグ修正を担当

経験した技術

- Redux ベースのアーキテクチャ（VueFlux）での状態管理
- デザインシステムに沿った UI の実装

取り組み・貢献

- デザイナー・Android・Web フロントのエンジニアと連携し、プラットフォーム間で仕様に差が出ないよう調整

開発環境

- **Swift**
 - アーキテクチャ: Redux
 - Swift 標準 SDK & API: UIKit , AVFoundation
 - サードパーティ製 SDK: Carbon , VueFlux , ReactiveSwift , XcodeGen , Quick/Nimble , APIKit , CocoaPods , Carthage , Lottie
- **クラウド連携**
 - Firebase: Crashlytics
- **CI/CD**

- CircleCI , Fastlane
- プロジェクト管理
 - Wrike , Kibela
- インターフェース共有
 - Protocol Buffers , Swagger
- デザインツール
 - Figma

[No.5] ドローン制御アプリ開発（ドローンベンチャー B社）

チーム体制

- 案件全体人数：約15名
 - iOS エンジニア：6名（担当）

案件概要・担当業務

- BtoB 向けドローン制御アプリで、飛行の安定性改善・複数社ドローン対応・画面開発を担当
- アーキテクチャの検討、見積もり、実装、レビュー、バグ修正を担当

経験した技術

- Swift
 - アーキテクチャの検討
 - Clean Architecture での実装
 - SwiftUI・UIKit x Combine を用いた画面実装
 - Swift Concurrency を用いた非同期処理の実装
 - Firebase Crashlytics、Xcode Organizer を用いたバグの原因調査
 - Logger API を用いたログ出力
 - Quick/Nimble ライブラリを用いた可読性の高いテストコードの記述
 - Mock を活用したテストコードの記述
 - iPad サイズ対応のアプリの実装
- IoT
 - 外部ライブラリを用いたドローンの制御の Swift での実装
 - PID 制御などの制御工学に基づく適切な制御モデルの Swift 実装
 - ROS (Robot Operating System) 環境の活用

取り組み・貢献

- Clean Architecture の採用により、UI を変えずにライブラリを差し替え、レイヤーごとの独立したテストを実現
- UIKit と Delegate パターンの実装を SwiftUI・Combine・Swift Concurrency でリファクタリング
- チーム運用の改善
 - 見積会の実施
 - レトロスペクティブの実施
 - 開発チーム朝ハドル会の実施
 - プロダクトバックログに「Ready」（着手できる状態）の定義を導入
 - Pull Request 提出から Merge までの運用ルールの見直し
 - リリースブランチ運用の見直し
 - デイリー前の Slack リマインダーの設定
 - デイリーでの相談事項の事前エントリー制の導入
 - Firebase Crashlytics 運用の見直し

開発環境

- Swift
 - アーキテクチャ: VIPER , Clean Architecture
 - Swift 標準 SDK & API: SwiftUI , UIKit , Combine , Swift Concurrency , Logger , MetricKit
 - サードパーティ製 SDK: Realm , Quick/Nimble , APIKit , CocoaPods , Carthage
- クラウド連携

- Firebase: Crashlytics , Analytics
- **CI/CD**
 - Bitrise , Fastlane
- プロジェクト管理
 - Zenhub
- デザインツール
 - Figma

[No.4] ショッピングアプリ開発

チーム体制

- 案件全体人数 : 2名
 - Flutter エンジニア : 1名 (担当)
 - デザイナー : 1名

案件概要・担当業務

- ショッピングアプリのデモを Flutter で開発し、iOS・Android の両方に対応
- Flutter のフロントエンド実装から Firebase のバックエンド実装まで一人で担当

経験した技術

- **Flutter**
 - Provider による状態管理
- **Firebase**
 - Authentication による認証
 - Firestore によるデータの永続化、NoSQL DB 設計
 - Storage への画像データの永続化
 - Crashlytics によるクラッシュ報告管理
 - App Distribution による iOS・Android のアプリ配布
 - Analytics による KPI の集計
 - Google Maps API での地図活用
- **CI/CD**
 - Codemagic での iOS・Android のアプリ配布の自動化
 - Fastlane から App Distribution への配布

取り組み・貢献

- Firebase によるサーバーレス構成で、モバイルバックエンドの設計・実装を担当
- Flutter での Android 対応 (マテリアルデザイン、Google Play ストアでの配信)

開発環境

- **Flutter**
 - Provider
- **クラウド連携**
 - Firebase: Authentication , Firestore , Storage , Crashlytics , App Distribution , Analytics
 - Google Maps API
- **CI/CD**
 - Codemagic , Fastlane
- **デザインツール**
 - Adobe XD

[No.3] 飲食店管理アプリ開発

チーム体制

- 案件全体人数 : 約10名

- iOS エンジニア : 3名 (リードエンジニア担当)

案件概要・担当業務

- BtoB 向け飲食店管理モバイルアプリの MVP 開発
- iOS リードエンジニアとして、要件調整とサーバーサイドとの API インターフェース検討を担当
- Scrum Master も兼任

経験した技術

- **Swift**
 - SwiftUI での画面実装
 - Codable プロトコルを用いた JSON の変換
 - TestFlight によるアプリ配信
 - アーキテクチャ、ディレクトリ構成の検討
- **Scrum Master**
 - スクラムボードの設計
 - 会議のファシリテーション

取り組み・貢献

- 指定された要件の実装にとどまらず、顧客やデザイナーへ仕様・デザインの改善案を提案
- スクラムボードの運用やコードレビューを、レトロスペクティブを待たずチーム内で相談し改善

開発環境

- **Swift**
 - アーキテクチャ: MVVM
 - Swift 標準 SDK & API: SwiftUI
 - サードパーティ製 SDK: SwiftLint
- **プロジェクト管理**
 - Zenhub , Trello
- **デザインツール**
 - Adobe XD

[No.2] クレジットカードアプリ iOS 開発

チーム体制

- 案件全体人数 : 約20名
 - iOS エンジニア : 4-5名 (担当)

案件概要・担当業務

- BtoC 向けクレジットカードアプリで、見積もり・実装・テスト・レビュー・バグ修正を担当
- iOS チームの負荷が上がったときに兼任 (主担当はサーバーサイドの No.1 案件)

経験した技術

- UIKit での画面実装
- API 疎通
- Realm でのデータ永続化
- XCTest でのテストコード実装
- MVVM での実装
- Delegate パターンの実装
- Human Interface Guidelines に基づいた UI 実装
- Moneytree LINK SDK といったサードパーティー製のライブラリの組み込み

取り組み・貢献

- API 側の実装も見ていたため、不具合の原因が iOS と API のどちらにあるかを切り分け
- mitmproxy で実機の通信をキャプチャし、リクエストとレスポンスから API 通信の不具合を調査
- API の仕様を把握している立場から、Android チームの担当者にも接続の実装方針を助言

開発環境

- **Swift**
 - アーキテクチャ: MVVM
 - Swift 標準 SDK & API: UIKit
 - サードパーティ製 SDK: CocoaPods , Carthage , Realm , Moneytree LINK SDK
- **通信キャプチャ**
 - mitmproxy
- **プロジェクト管理**
 - Jira , Confluence , Trello
- **デザインツール**
 - Sketch , InVision

[No.1] クレジットカードアプリ API 開発

チーム体制

- 案件全体人数 : 約20名
 - サーバーサイドエンジニア : 4名 (担当)

案件概要・担当業務

- BtoC 向けクレジットカード明細管理アプリのリニューアルに伴い、API やバッチの開発
- サブリードエンジニアとして、顧客向け資料作成から設計・実装・テスト・レビューまでを担当
- チーム結成から約 2 年半、リリース後のシステムを継続的にアップデート

経験した技術

- **API 設計/開発**
 - PHP での API 設計・開発
 - MySQL での DB 設計・開発
 - OAuth2.0 での認証・認可の実装
- **バッチ設計/開発**
 - Java でのバッチの設計・開発
- **テスト**
 - API の単体・結合テストの設計と実装
 - Postman での API テストの自動化
 - JMeter での負荷テスト
- **ドキュメンテーション**
 - OpenAPI (Swagger) でのインターフェース設計・共有
 - PlantUML での設計

取り組み・貢献

- iOS チームとの兼任を活かし、モバイル視点を API インターフェース設計に反映
- API の結合テストを Postman で自動化し、少ない工数で網羅的な繰り返しテストを実施

開発環境

- **使用言語**
 - PHP (CodeIgniter)
 - Java
- **プロジェクト管理**
 - Jira , Confluence , Trello
- **テストツール**
 - Postman , JMeter
- **ドキュメンテーション**
 - OpenAPI , PlantUML , draw.io

技術スタック

AI

- AI アプリケーション開発
 - モデル: GLM , Kimi , gpt-oss , DeepSeek , bge-m3 (Embedding)
 - RAG / 検索: Cloudflare Vectorize , Workers AI
 - MCP: @modelcontextprotocol/sdk v2 , Better Auth OAuth , WebMCP
 - 実行基盤: Cloudflare AI Gateway , Cloudflare Workflows , Cloudflare Queues , Cron Triggers
- AI コーディング / エージェント運用
 - コーディングエージェント: Claude Code , Codex , Antigravity
 - 定期実行 / 自動化: Claude Code Routines , Codex Automation
 - エージェント拡張: Skills , MCP , Chrome DevTools MCP
 - プロトタイピング: v0 , Claude Design

フロントエンド / モバイル

- Web アプリ開発
 - フレームワーク: React , Next.js , Astro , TanStack Start
 - ルーティング / データ取得: TanStack Router , TanStack Query
 - UI: shadcn/ui , Tailwind CSS
- SEO / パフォーマンス
 - 計測・改善: Lighthouse , PageSpeed Insights , Google Search Console
 - 実装: 構造化データ・OGP 自動生成・ sitemap ・ llms.txt 、 Pagefind によるサイト内全文検索
- 分析 / 計測
 - プロダクト分析: PostHog , Looker Studio , Adjust , BytePlus DI
 - アクセス解析: Cloudflare Analytics , Firebase Analytics
- モバイルアプリ開発
 - クロスプラットフォーム: React Native (Expo) , Flutter (Dart)
 - ネイティブ: Swift (SwiftUI, UIKit)
- アーキテクチャ
 - 設計パターン: Clean Architecture , レイヤードアーキテクチャ , MVVM , Redux , Riverpod
 - モノレポ: Bun workspaces
 - 独自アーキテクチャ: 「[Framework-Independent Architecture \(FIA\)](#)」の考案・公開 ([スライド](#) / [YouTube](#))

バックエンド

- サーバーサイド
 - API 設計: REST API , GraphQL
 - 実装: TypeScript , Bun , Hono , oRPC , Node.js , Python , PHP , Java
 - 外部 API 連携: LINE Messaging API , Slack API , Google Drive API , GitHub App
- データベース
 - 設計: RDB テーブル設計
 - ORM: Prisma , Drizzle
 - RDBMS: PostgreSQL , MySQL , SQLite (Cloudflare D1)
 - NoSQL / ベクトル DB: Firestore , Cloudflare Vectorize
- 認証 / 認可
 - プロトコル / 仕様: OAuth 2.0 , JWT
 - 実装: Better Auth
 - サービス: LINE Login , Cognito , Firebase Authentication , Cloudflare Access
 - クラウド間連携: Workload Identity Federation (OIDC)

クラウド / インフラ

- Cloudflare

- コンピュート / 配信: `Workers` , `Pages` , `Browser Rendering`
- AI: `Workers AI` , `AI Gateway` , `Vectorize`
- データ / 非同期処理: `D1` , `R2` , `KV` , `Queues` , `Workflows`
- ネットワーク / セキュリティ: `DNS` , `Access` , `Turnstile` , `Email Routing` , `Email Sending`
- **Google Cloud**
 - `Cloud Run` , `GCS` , `IAM` , `Cloud Identity` , `Workload Identity Federation`
- **AWS**
 - `Organizations` , `IAM Identity Center` , `Amplify` , `AppSync` , `Cognito` , `S3` , `Route 53`
- **Firebase**
 - `Authentication` , `Firestore` , `Storage` , `Analytics` , `Crashlytics` , `App Distribution` , `Remote Config`
- **IaC**
 - `Terraform` , `tflint` , `dotenvx` , `OIDC`

品質 / CI・CD

- **テスト**
 - ユニット / 結合: `Vitest` , `Swift Testing` , `XCTest` , `Quick/Nimble`
 - E2E: `Playwright` , `Maestro`
 - UI / モック / アクセシビリティ: `Storybook` , `MSW` , `@axe-core/playwright`
- **コード品質**
 - Lint / フォーマット: `Biome` , `oxlint` , `oxfmt` , `ESLint` , `Prettier`
 - 依存 / 構成の検査: `knip` , `dependency-cruiser`
 - コミット / Git フック: `commitlint` , `lefthook` , `husky`
 - 依存更新: `Renovate`
- **CI/CD**
 - CI: `GitHub Actions` , `Xcode Cloud` , `Bitrise`
 - リリース / デプロイ: `release-please` , `Wrangler`

OSS 開発 (自作 CLI ツール)

- [ai-chat-md-export](#) (AI チャット履歴の Markdown 変換)
- [mermaid-markdown-wrap](#) (Mermaid の Markdown ラップ)
- [issue-linker](#) (GitHub Issue 参照の検証)
- [readme-i18n-sentinel](#) (README 翻訳の構造検証)